МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:

А.М. Литвинчук

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

Лабораторная работа 7

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

1) Изложение цели работы.
2) Задание по лабораторной работе с описанием своего варианта.
3) Спецификации ввода-вывода программы.
4) Текст программы (кратко).
5) Выводы по проделанной работе.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

12) Задать составление строки из символов, появляющихся из разных углов формы и выстраивающихся друг за другом. Процесс должен циклически повторяться.

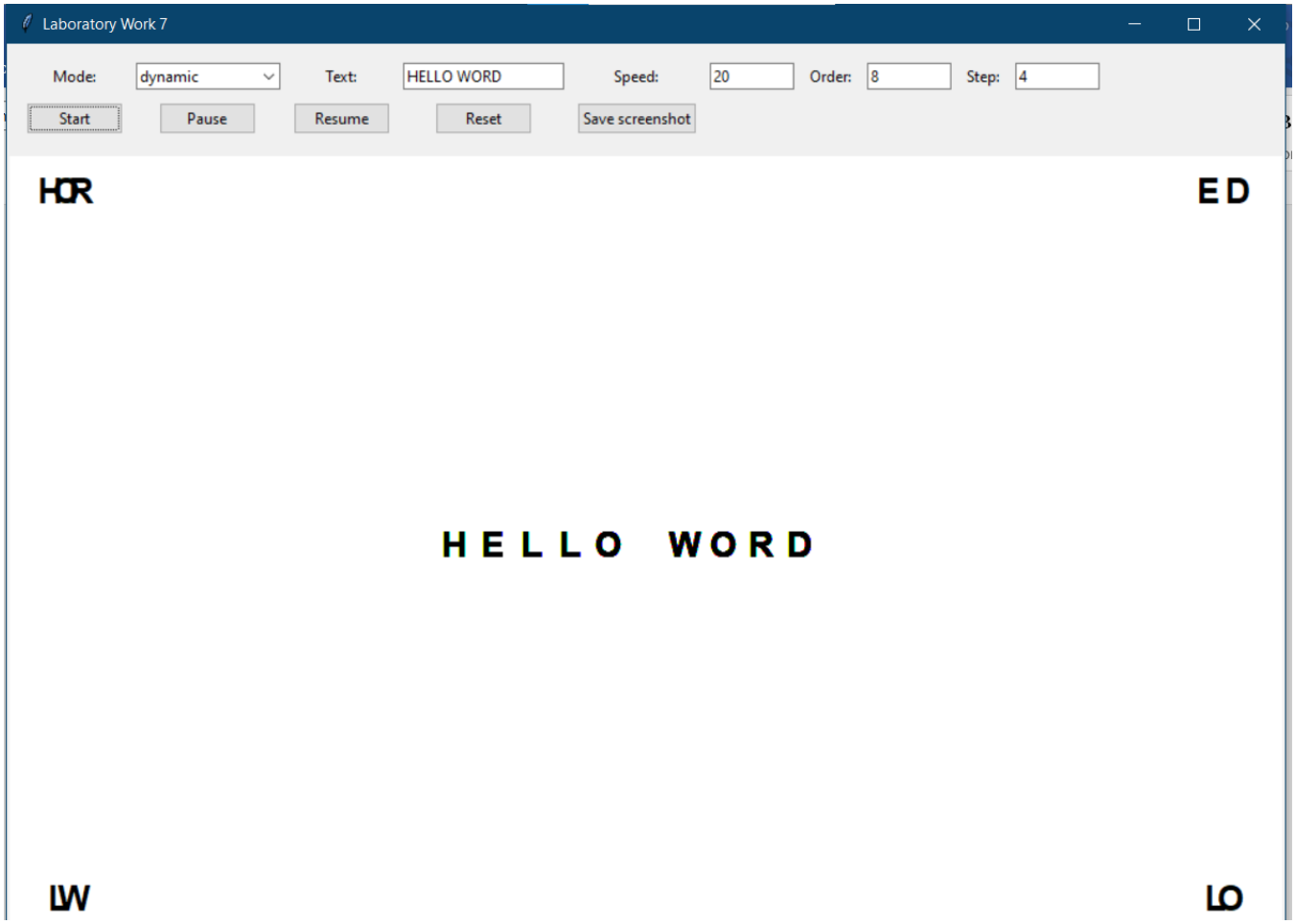
```python
"""Main application for laboratory work 7."""

import tkinter as tk
from tkinter import messagebox, ttk

from src.dynamic_text import FlyingLetter, LetterTarget
from src.hilbert import hilbert_curve_points
from src.screenshot import save_widget_screenshot


class Lab7App:
    """Main GUI application for dynamic text and Hilbert curve."""

    def __init__(self, root):
        """Initialize the application."""
        self.root = root
        self.root.title("Laboratory Work 7")
        self.root.geometry("1000x700")

        self.running = False
        self.paused = False
        self.letters = []
        self.animation_after_id = None

        self.variables = {
            "mode": tk.StringVar(value="dynamic"),
            "text": tk.StringVar(value="HELLO"),
```

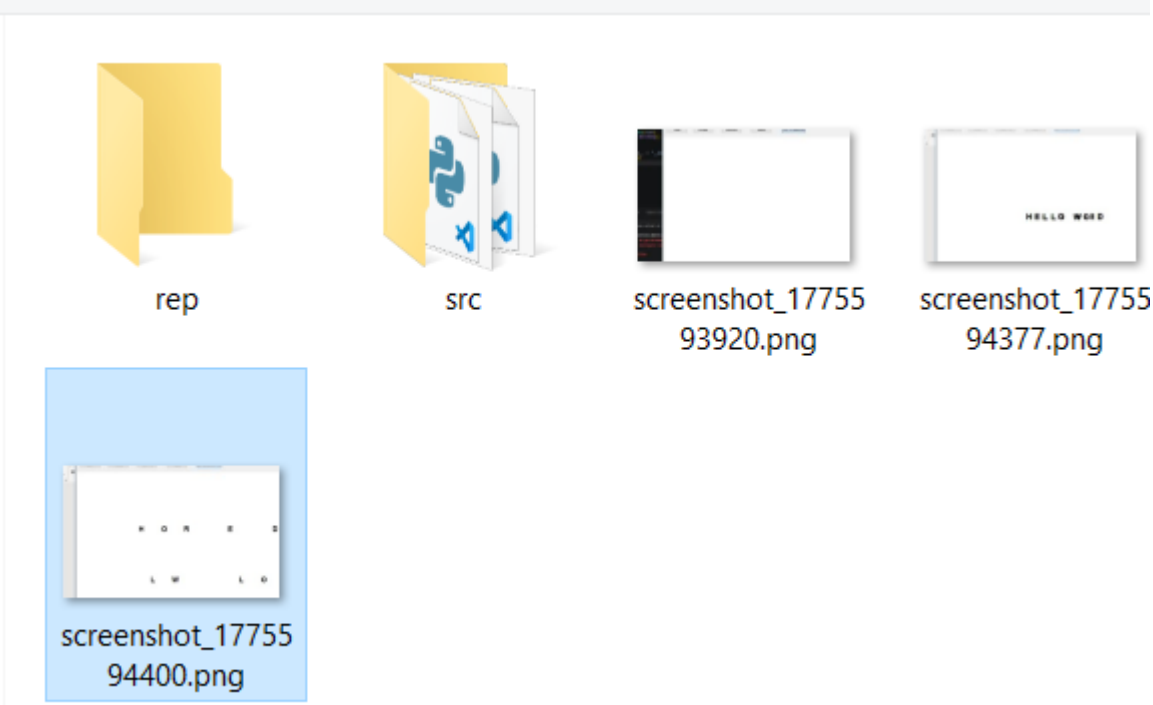
```
            "speed": tk.StringVar(value="20"),
            "order": tk.StringVar(value="4"),
            "step": tk.StringVar(value="10"),
        }

        self.canvas = None
        self.create_widgets()

    def create_widgets(self):
        """Create all interface widgets."""
        control_frame = ttk.Frame(self.root, padding=10)
        control_frame.pack(side=tk.TOP, fill=tk.X)

        ttk.Label(control_frame, text="Mode:").grid(row=0, column=0, padx=5, pady=5)
        ttk.Combobox(
            control_frame,
            textvariable=self.variables["mode"],
            values=["dynamic", "hilbert"],
            state="readonly",
            width=15,
        ).grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(control_frame, text="Text:").grid(row=0, column=2, padx=5, pady=5)
        ttk.Entry(
            control_frame,
            textvariable=self.variables["text"],
            width=20,
        ).grid(row=0, column=3, padx=5, pady=5)

        ttk.Label(control_frame, text="Speed:").grid(row=0, column=4, padx=5, pady=5)
        ttk.Entry(
            control_frame,
            textvariable=self.variables["speed"],
            width=10,
        ).grid(row=0, column=5, padx=5, pady=5)

        ttk.Label(control_frame, text="Order:").grid(row=0, column=6, padx=5, pady=5)
        ttk.Entry(
            control_frame,
            textvariable=self.variables["order"],
            width=10,
        ).grid(row=0, column=7, padx=5, pady=5)

        ttk.Label(control_frame, text="Step:").grid(row=0, column=8, padx=5, pady=5)
        ttk.Entry(
            control_frame,
            textvariable=self.variables["step"],
            width=10,
        ).grid(row=0, column=9, padx=5, pady=5)

        ttk.Button(control_frame, text="Start", command=self.start).grid(row=1, column=0,
padx=5, pady=5)
        ttk.Button(control_frame, text="Pause", command=self.pause).grid(row=1, column=1,
padx=5, pady=5)
```

```
        ttk.Button(control_frame, text="Resume", command=self.resume).grid(row=1, column=2,
padx=5, pady=5)
        ttk.Button(
            control_frame,
            text="Reset",
            command=self.reset_canvas,
        ).grid(row=1, column=3, padx=5, pady=5)
        ttk.Button(
            control_frame,
            text="Save screenshot",
            command=self.save_screenshot,
        ).grid(row=1, column=4, padx=5, pady=5)

        self.canvas = tk.Canvas(self.root, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)

    def reset_canvas(self):
        """Clear the canvas and stop current animation."""
        self.running = False
        self.paused = False
        self.letters.clear()

        if self.animation_after_id is not None:
            self.root.after_cancel(self.animation_after_id)
            self.animation_after_id = None

        self.canvas.delete("all")

    def pause(self):
        """Pause the animation."""
        self.paused = True

    def resume(self):
        """Resume the animation."""
        self.paused = False

    def save_screenshot(self):
        """Save a screenshot of the canvas."""
        filename = save_widget_screenshot(self.canvas)
        messagebox.showinfo("Saved", f"Screenshot saved:\n{filename}")

    def start(self):
        """Start the selected mode."""
        self.reset_canvas()

        if self.variables["mode"].get() == "dynamic":
            self.start_dynamic_text()
        else:
            self.draw_hilbert()

    def start_dynamic_text(self):
        """Start dynamic text animation."""
        text = self.variables["text"].get()
```

```
        if not text:
            messagebox.showerror("Error", "Enter text")
            return

        try:
            speed = float(self.variables["speed"].get())
            if speed <= 0:
                raise ValueError("Speed must be positive")
        except ValueError:
            messagebox.showerror("Error", "Speed must be a positive number")
            return

        self.running = True
        width = self.canvas.winfo_width() or 1000
        height = self.canvas.winfo_height() or 600

        corners = [
            (0, 0),
            (width, 0),
            (0, height),
            (width, height),
        ]

        start_x = width // 2 - len(text) * 15
        start_y = height // 2

        self.letters = []
        for index, char in enumerate(text):
            start_position = corners[index % 4]
            target = LetterTarget(
                x_coord=start_x + index * 30,
                y_coord=start_y,
            )
            self.letters.append(FlyingLetter(self.canvas, char, start_position, target))

        self.animate_letters(speed)

    def animate_letters(self, speed):
        """Animate letters until all reach their targets."""
        if not self.running:
            return

        if self.paused:
            self.animation_after_id = self.root.after(
                30,
                lambda: self.animate_letters(speed),
            )
            return

        completed = 0
        for letter in self.letters:
            if letter.move_step(speed):
                completed += 1
```

```python
        if completed == len(self.letters):
            self.animation_after_id = self.root.after(
                1000,
                self.start_dynamic_text,
            )
        else:
            self.animation_after_id = self.root.after(
                30,
                lambda: self.animate_letters(speed),
            )

    def draw_hilbert(self):
        """Draw the Hilbert curve."""
        try:
            order = int(self.variables["order"].get())
            step = int(self.variables["step"].get())
            if order < 1 or step <= 0:
                raise ValueError("Invalid order or step")
        except ValueError:
            messagebox.showerror(
                "Error",
                "Order must be >= 1 and step must be > 0",
            )
            return

        points = hilbert_curve_points(order, step, 50, 600)
        for index in range(len(points) - 1):
            x1_coord, y1_coord = points[index]
            x2_coord, y2_coord = points[index + 1]
            self.canvas.create_line(
                x1_coord,
                y1_coord,
                x2_coord,
                y2_coord,
            )


def main():
    """Run the application."""
    root = tk.Tk()
    Lab7App(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```





Вывод: В ходе лабораторной работы были изучены возможности Python для разработки оконных приложений. Была создана программа с графическим интерфейсом, реализующая анимацию символов и построение кривой Гильберта с возможностью изменения параметров, приостановки выполнения и сохранения скриншотов. В результате были получены практические навыки работы с графикой, анимацией и пользовательским интерфейсом в Python.